Translation AOO

Original author: Mechtilde Stehmann

Contents

Chapter 1 4
Introduction 4
1.1 Prerequisites 4
1.2 Languages 4
Chapter 2 5
Build Environment 5
Chapter 3 7
Extracting the SDF file "en-US" from the code 7
Chapter 4 8
Common Components of the Scripts 8
4.1 License Text 8
4.2 Log File 8
4.3. POOTLE environment 9
4.4 Import the LanguageLists.sh script 9
Chapter 5 9
From the SDF file to the Pootle database 9
5.1 UpdateTranslation.sh – Header 9
5.2 Cleaning Up the Working Directories 10
5.3 Template Creation and Import into the Pootle Database 10
5.4 Synchronizing the individual languages with the current templates 11
Chapter 6 14
Nightly Tasks 14
6.1 Extracting from Pootle 14
6.2 Copy to the home directory 15
Chapter 7 16
PO Files to SDF Files 16
7.1 Header Text 16
7.2 Script for Generating SDF Files 17
Chapter 8 18
Running gsicheck 18
Chapter 9 19
Performing Individual Steps 19
9.1 sdf to pot 19
9.2 Importing POT Files into the Database 19
9.3 Backing Up the Existing Translation 20
9.4 Synchronizing the translations with the source 20
9.5 Update the Database 21
9.6 Updating the Working Directory 21
Chapter 10 21
Rarely Used Scripts 22
10.1 Creating Language Directories 22
10.2 Recovery After a Failure 22
10.3 Restoring the PO Files 23

Chapter 11.....**24**
Extraction Scripts.....24
11.1 Extracting the Documentation.....24
11.2 Extraction of the documented scripts.....24
Chapter 12.....**25**
Scripts for Automation.....25
12.1 Pootle.conf.....25
References.....**29**

Chapter 1

Introduction

The necessary processes to make localized versions of Apache OpenOffice are developing into a comprehensive project. This will be described in this documentation.

The necessary steps will be outlined below. These steps are performed at various points in the process.

1.1 Prerequisites

First, a virtual machine is required on which the Pootle software is running. This software can be downloaded from http://docs.translatehouse.org/pr

The virtual machine is provided on the ASF infrastructure and is accessible on the web at

https://translate.apache.org/projects/.

Access via SSH is achieved using

ssh <user>@translate-ec2-va.apache.org

A machine is also needed to build *Apache OpenOffice*. A Buildbot is available for this purpose at **https://ci2.apache.org/#/builders/**.

From there, the generated *en-US* version is made available for download at **https://nightlies.apache.org/openoffice/install/linsnap-42x/**.

1.2 Languages

Processes used to provide localized versions of *Apache OpenOffice* are, as far as possible, implemented in (shell) scripts and automated.

Many scripts iterate over a list of languages to be processed. The corresponding lists are provided and maintained in a separate (script) file, which is included by the respective scripts.

⟨LanguageLists.sh 2⟩

```
#!/bin/bash
# All Languages
LANGS_ALL="aa ab af am ar ast bg ca ca_XR ca_XV cs da de el en_GB es et \
eu fa fi fr gd gl he hi hr hu hy id is it ja kab kk km kn ko ky lt lv nb \
nl nn om pl ps pt pt_BR ro ru sh si sk sl so sr sv ta th tlh tr tt_RU ug \
uk vi zh_CN zh_TW"

LANGS_INCOMPLETE="af am ar ast ca_XV cs da el es eu fa gd gl he hi hr hu \
id is it kab kk km kn ko ky lv nb nn om pl ps pt pt_BR ro ru si sk sl \
so sr sv ta th tlh tt_RU ug uk vi zh_CN zh_TW"

# LANGS="ca_XR ca_CV en_GB pt_BR tt_RU zh_CN zh_TW"
```

```
# Completely translated UI
LANGS_UI="bg ca ca_XR de en_GB et fi fr hy ja lt nl tr"
# Completely translated HELP
LANGS_HELP="de en_GB fr ja nl"
# LANGS=${LANGS_ALL}
LANGS="de uk”
```

Chapter 2

Build Environment

Occasionally, it is useful to set up your own build environment.

To do this, clone the corresponding Git repository. On the work computer, the cloned Apache OpenOffice Git repository is located in the */home/<user>/openoffice* directory.

To build, execute the following commands one after another:

- cd /home/<user>/openoffice
- cd main
- dmake clean
- autoconf
- ../../configuration/configure.sh
- ./bootstrap
- source LinuxX86-64EnvSet.sh
- cd instsetoo_native
- •build --all -P4 -- -P4

The following shows the `configure` script I used.

⟨configure.sh⟩

```
#!/bin/bash
LOG=${HOME}/log_$(date +%Y%m%d%H%M%S).txt
touch ${LOG}

#cd ${HOME}/openoffice
# git checkout AOO42XMS
#GitHash=$(git log -n1 | grep commit | cut -d ' ' -f2)
#echo $GitHash >>${LOG}

cd ${HOME}/openoffice/main

# Include the language data
. ${HOME}/configuration/langs.sh
echo ${LANGS} >>${LOG}

./configure \
--with-dmake-path=/usr/local/bin/dmake \
--with-epm-url="https://github.com/jimjag/epm/archive/v5.0.0/epm-5.0.0.tar.gz" \
```

```
--with-system-python \
--with-lang="${LANGS}" \
--with-jdk-home=/usr/lib/jvm/java-8-openjdk-amd64 \
--with-package-format="deb rpm" \
--with-vendor="Mechtilde" \
--with-build-version="aoo42x-$(date +"%Y-%m-%d %H:%M") -`uname -sm`\\\\n $
{GitHash}" \
--with-system-stdlibs \
--enable-crashdump=yes \
--enable-category-b \
--enable-beanshell \
--enable-wiki-publisher \
--enable-bundled-dictionaries \
--enable-report-builder \
--enable-opengl \
--enable-dbus \
--without-junit \
--without-stlport \
--disable-odk \
--without-fonts \
>>${LOG} 2>&1

# --with-epm=/usr/local/bin/epm
# --with-epm-url="https://github.com/jimjag/epm/archive/v5.0.0/epm-5.0.0.tar.gz
# --enable-report-builder enables the build of the Report Builder extension
# --enable-mysql-connector enables the build of the MySQL Connector/OOo extension.
#                                This requires access to the MySQL Connector/C (also
                                 known as libmysql)
#                                must also be provided, either with the
# --with-system-mysql      Use MySQL libraries already on the system to build
                                 the MySQL Connector/OOo extension. Requires
#                                MYSQLCONFIG to point to the mysql_config executable.
#                                or
# --with-libmysql-path     Use the Connector/C (libmysql) installation to build
                                 the MySQL # Connector/OOo extension.
                                 Usage: --with-libmysql-path=<absolute path to your
                                                              Connector/C installation>
# --with-system-mysql-cppconn Use MySQL C++ Connector libraries already on the system

# --enable-report-builder \
# --enable-mysql-connector \
# --with-system-mysql \
# --with-libmysql-path=/usr/include/mysql++ \
# --with-system-mysql-cppconn \
# --with-java=/usr/lib64/openjdk-11 \
# EPM version 4.4.2 instead of 4.2 from the repository
# --with-epm=/usr/local/bin/epm \
# --enable-gstreamer
```

After that the following commands were executed:

```
./bootstrap >> ${LOG}
source LinuxX86-64Env.Set.sh
cd instsetoo_native
build --all -P4 -- -P4
```

Chapter 3

Extracting the SDF file "en-US" from the code

To extract this file, you need a working build environment and a current build. The build process is described in Chapter 2 (page 3).

localize is part of the Apache OpenOffice source code and is built from it. In the *main* directory or a subdirectory, the command

```
localize -e -l en-US -f en-US.sdf
```

is executed. The main/* directory is part of the Git repository

- *-e* enables extraction mode.
- *-l* specifies the language(s) to be extracted using the ISO code. Multiple languages are separated by commas.
- *-f* specifies the output file.

The `*localize*` function also includes a `*gsicheck*` (Chapter 8, page 37).

The generated file `*en-US.sdf*` is located in the `*main*` directory.

If you have your own build environment, you can use `*scp*`.

```
<user>@<name of the build server>:~$ scp -p \
/home/<user>/openoffice/en-US.sdf \
<user>@translate-ec2-va.apache.org:/home/<user>/aoo/workdir/
```

At **https://ci2.apache.org/#/builders**, there is a build system running on Linux. This is operated as a Buildbot.

On the Buildbot, after a successful build for version 4.2.x, the file *en-US.sdf* is also created. This file is stored at

https://nightlies.apache.org/openoffice/install/linsnap-42x/en-US.sdf

and can be downloaded from there to the machine **translate-ec2-va.apache.org.**

To do this, a cron job for the download is set up on the Pootle server. It runs weekly.

```
# m h dom mon dow command
0 1 * * 6 /home/mechtilde/local/bin/Download.sh
```

The cron job runs the following script.

⟨Download.sh⟩

```
#!/bin/bash
cd ${HOME}
wget https://nightlies.apache.org/openoffice/install/linsnap-42x/en-US.sdf.
```

Chapter 4

Common Components of the Scripts

The following scripts are intended for use on the Pootle server.

4.1 License Text

All scripts begin with the license notice. The Apache-2.0 license is used.

```
#
# License: Apache-2.0
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements.
# The ASF licenses this file to you under the Apache License,
# Version 2.0 (the "License"); you may not use this file except
# in compliance with the License. You may obtain a copy of the
# License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, whether express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.

# Authors: Michael Stehmann <mikeadvo@apache.org >
# Mechtilde Stehmann <mechtilde@apache.org>
```

4.2 Log File

The scripts record their execution and error messages in a log file. The name of the log file is the name of the respective script with the suffix *.log*.

```
# Logging error messages
LOG_PATH=${HOME}/log
LOG_FILE=${LOG_PATH}/${ScriptName}"-"$(date +%Y-%m-%d-%H%M%S).log
exec 2>>${LOG_FILE}
echo "Messages from "$0 >>${LOG_FILE}
```

4.3. POOTLE environment

Current versions of Pootle run in a "*virtual environment*." The required Python(2) interpreter is also located there.

The script assigns the necessary paths to the "*virtual environment*," the directory containing the translations, and the user's working directory to variables.

```
# Newer Pootle versions run inside a virtual environment
VIRTUAL_ENV=/x1/www/pootle/env
POOTLE_SETTINGS=${VIRTUAL_ENV}/pootle.conf
PYTHONPATH=${VIRTUAL_ENV}/lib/python2.7/site-packages
PATH="$VIRTUAL_ENV/bin:$PATH"

export VIRTUAL_ENV
export PATH
export POOTLE_SETTINGS
export PYTHONPATH

PROJECT_PATH=/x1/www/po

# Directory of the pootle command and Python 2
POOTLE_DIR=${VIRTUAL_ENV}/bin

# Work directory
WORK_DIR=${HOME}/aoo/workdir
```

4.4 Import the LanguageLists.sh script

The file containing the languages to be edited is imported (see Chapter 1.2, page 2).

```
# Include the language data
. ${HOME}/local/bin/LanguageLists.sh
echo ${LANGS}
echo ${LANGS} >>${LOG_FILE}
```

Chapter 5

From the SDF file to the Pootle database

The `UpdateTranslation.sh` script performs all the steps necessary to update the Pootle database based on the SDF file extracted from the source code. This process also updates the POT and PO files.

5.1 UpdateTranslation.sh – Header

The beginning of the script contains all common components (Chapter 4, page 9).

⟨UpdateTranslation.sh⟩

```
#!/bin/bash
ScriptName="UpdateTranslation" #(.sh)
# Version 0.1.0
# This file should be in ~/local/bin/.

TEMPLATES_DIR=/x1/www/po/aoo40/templates
```

5.2 Cleaning Up the Working Directories

The working directory *$WORK_DIR* is *$HOME/aoo/workdir.* The following steps are performed there:

- Check for the existence of the (current) en-US.sdf (as a precaution)
- Removing the old POT files
- Recreate the directories for the POT files

⟨UpdateTranslation⟩

```
# If the (non-empty) file en-US.sdf is in ${HOME}:
if [ -s ${HOME}/en-US.sdf ]
then
mv ${HOME}/en-US.sdf ${WORK_DIR}/en-US.sdf

# To be carefull
rm -r ${WORK_DIR}/POT-UI/
rm -r ${WORK_DIR}/POT-HELP/helpcontent2

mkdir -p ${WORK_DIR}/POT-UI
mkdir -p ${WORK_DIR}/POT-HELP
```

5.3 Template Creation and Import into the Pootle Database

To do this, version 2.2.5 of the *Translate Toolkit* —which is installed in the virtual environment—is currently being used on the Pootle server.

- Convert *en-US.sdf* to POT files
- Move the *helpcontent2* directory to separate the help texts (HELP) from the user interface texts (UI)

```
VIRTUAL_ENV=/x1/www/pootle/env
POOTLE_DIR=${VIRTUAL_ENV}/bin
```

The conversion from SDF to POT is performed using *oo2po* from the *Translate-Toolkit* package. The *--pot* switch is used to create the directories containing the POT files.

On the Pootle server, the user interface texts and the help texts are two separate projects. Therefore, the help texts must be separated.

⟨UpdateTranslation⟩

```
# Convert en-US.sdf to templates
${POOTLE_DIR}/python ${POOTLE_DIR}/oo2po --progress=verbose \
--pot ${WORK_DIR}/en-US.sdf ${WORK_DIR}/POT-UI/

# Move helpcontent2 to POT-HELP to separate HELP from UI
mv ${WORK_DIR}/POT-UI/helpcontent2 ${WORK_DIR}/POT-HELP/
```

- Copying the files to the project directory

```
PROJECT_PATH=/x1/www/po
WORK_DIR=${HOME}aoo/workdir
```

⟨UpdateTranslation⟩

```
# Copy PotFiles-* to the project directory
cp --recursive --verbose ${WORK_DIR}/POT-UI/* \
${PROJECT_PATH}/aoo40/templates/
cp --recursive --verbose ${WORK_DIR}/POT-HELP/* \
${PROJECT_PATH}/aoo40help/templates/
```

The following steps use commands from the *Pootle* package[1] .

```
VIRTUAL_ENV=/x1/www/pootle/env
POOTLE_DIR=${VIRTUAL_ENV}/bin
```

- Updating the template entries in the database

After new or updated files (in this case, the templates—.pot files) have been uploaded to the Pootle server, they must be added to the database so that, for example, the current translation status is visible.

This is done separately for the user interface and help templates.

⟨UpdateTranslation⟩

```
# Update the translation templates into the Pootle database
${POOTLE_DIR}/python ${POOTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40 -language=templates

${POOTLE_DIR}/python ${POOTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40help --language=templates
```

1 https://pootle.translatehouse.org/

5.4 Synchronizing the individual languages with the current templates

For each language, the user interface files are processed first, followed by the help files.

The program's progress is documented in the log file.

While `*pootle update_stores*` synchronizes the database contents with the contents of the `po` and `*pot*` files, `*pootle sync_stores*` writes the data from the database to the `*po*` files in the Pootle server's file system.

⟨UpdateTranslation⟩

```
# Reconcile languages with templates
for Language in ${LANGS}
do
echo "Sync_stores UI "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
sync_stores --project=aoo40 -language=${Language}


mkdir -p ${WORK_DIR}/PO-Files/${Language}/
cd ${WORK_DIR}/PO-Files/

cp --recursive ${PROJECT_PATH}/aoo40/${Language} .
cp --recursive ${PROJECT_PATH}/aoo40/${Language} ../aoo40

cp --recursive ${PROJECT_PATH}/aoo40help/${Language}/helpcontent2 \
./${Language}
cp --recursive ${PROJECT_PATH}/aoo40help/${Language} \
../aoo40help/

# Commands for updating Git repository
cd ${WORK_DIR}
git add .
git commit -am "update $(date) ${Language}"
```

Pomerge is a tool from the *Translate-Toolkit* package.

Pomerge inserts corrected PO files (or snippets) into existing PO files.

Pomerge also tries to make the text changes as minimal as possible so that you can more easily identify the changes in your version control system.

Usage:

```
pomerge [options] [-t <template>] -i <input> -o <output>
```

template is a set of reference files; in this case, a set of *POT* files

input contains the corrected files that will overwrite the contents of *output*. These files are taken from the user's working directory.

output contains the files whose contents are to be overwritten by input.

These are the project files.

⟨UpdateTranslation⟩

```
echo "Pomerge UI "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pomerge \
--template=${PROJECT_PATH}/aoo40/templates/ \
--input ${WORK_DIR}/aoo40/${Language}/ \
--output ${PROJECT_PATH}/aoo40/${Language}/
```

The contents of the updated files are now imported into the database. They are then copied to the user's working directory to update it.

⟨UpdateTranslation⟩

```
echo "Update_stores UI "${Language}
>>${LOG_FILE} echo $(date +%Y-%m-%d-%H%M%S)
>>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40 --language=${Language}

cp --recursive ${PROJECT_PATH}/aoo40/${Language} ${WORK_DIR}/aoo40/
```

The procedures just described also applies to the help texts.

⟨UpdateTranslation⟩

```
echo "Sync_stores HELP "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
sync_stores --project=aoo40help --language=${Language}

echo "Pomerge HELP "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pomerge \
--template=${PROJECT_PATH}/aoo40help/templates/ \
--input ${WORK_DIR}/aoo40help/${Language}/ \
--output ${PROJECT_PATH}/aoo40help/${Language}/

echo "Update_stores HELP "${Language} >>${LOG_FILE}
```

```
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40help --language=${Language}

cp --recursive ${PROJECT_PATH}/aoo40help/${Language}/helpcontent2 \
${WORK_DIR}/aoo40help/${Language}/
```

At the end of each language pass, the Git repository is updated. Each pass results in a *commit*.

⟨UpdateTranslation⟩

```
# Commands for updating Git repository
cd ${WORK_DIR}
git add .
git commit -am "update $(date) ${Language}"

echo "Finished "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

done
fi
```

Chapter 6

Nightly Tasks

The following script, which simply calls other scripts, is executed every night at 2:00 a.m. (UTC) via a cron job.

⟨mnrun.sh⟩

```
#!/bin/bash

/home/mechtilde/local/bin/sync-db2files.sh 2>/dev/null
#echo "Finished syncing database to files"
/home/mechtilde/local/bin/cp2home.sh
#/home/mechtilde/local/bin/mksdf.sh
```

6.1 Extracting from Pootle

The translation results are continuously updated in the Pootle database via the web application.

The script transfers the changes from the database to the PO-files.

⟨sync-db2files.sh⟩

```
#!/bin/bash
ScriptName="sync-db2files" #(.sh)
# Version 0.1.0
```

```
# This file should be in ~/local/bin/.

# This script synchronizes the Pootle database to the file system
# for aoo42 and/or aoo42help.

for Language in
${LANGS[*]} do

echo ${Language} >>${LOG_FILE}

# When the --overwrite option is specified, the sync operation will not be
# conservative and will overwrite the existing files on disk, making
# strings obsolete and updating the file's structure.

echo "UI" >>${LOG_FILE}
${VIRTUAL_ENV}/bin/python ${VIRTUAL_ENV}/bin/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
sync_stores --overwrite --project=aoo40 -language=${Language}

echo "HELP" >>${LOG_FILE}
${VIRTUAL_ENV}/bin/python ${VIRTUAL_ENV}/bin/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
sync_stores --overwrite --project=aoo40help --language=${Language}

echo "Done "${Language} >>${LOG_FILE}
done
echo "Done all"${date} >>${LOG_FILE}
```

6.2 Copy to the home directory

The following script copies the PO files from the corresponding subdirectories of the */x1/www/po* directory to the corresponding subdirectories of the user's home directory.

If any language subdirectories do not exist there, they are created. The script also adds new files to the Git repository and creates a commit.

⟨cp2home.sh⟩

```
#!/bin/bash
ScriptName="cp2home" #(.sh)

# This script copies the *.po files
# from aoo40 and aoo40help to the home directory

# Work directory
WORK_DIR=${HOME}/aoo/workdir

PROJECT_PATH=/x1/www/po

for Language in ${LANGS[*]}
do

# Language2=$(echo ${Language} | sed -e 's/_/-/1')
# mkdir -p ${WORK_DIR}/SDF-Files/${Language2}/

mkdir -p ${WORK_DIR}/PO-Files/${Language}/
```

```
cd ${WORK_DIR}/PO-Files/
cp --recursive ${PROJECT_PATH}/aoo40/${Language} .
cp --recursive ${PROJECT_PATH}/aoo40/${Language} ...../aoo40

cp --recursive ${PROJECT_PATH}/aoo40help/${Language}/helpcontent2 \
./${Language}
cp --recursive ${PROJECT_PATH}/aoo40help/${Language} \
../aoo40help/

done

# Commands for updating Git repository
cd ${WORK_DIR}
git add .
git commit -am "update $(date) PO files"
```

Chapter 7

PO Files to SDF Files

This only works on a current ASF machine

On the VM provided by the ASF, there is a copy of the PO files extracted from the database in a Git repository located in the home directory.

This is currently still being processed on a production machine.

To do this, the cloned Git directory

```
mechtilde@translate-ec2-va.apache.org:/home/mechtilde/aoo/workdir
```

is updated.

Afterward, a script is used to generate an updated SDF file from the downloaded PO files.

7.1 Header Text

To do this, there must be a file named *header.txt* in the directory that contains the license text for the SDF files to be generated.

⟨header.txt⟩

```
#
# ####### # # ### ##### ##### #### ##### ##### # # # #
# ## # # # # # # # # # # # # #
# # # # # # # #### # # #
# # # # # # ## # # # # # # #
# #### ### # # ### # ##### #### ##### #
#
# DO NOT EDIT! This file will be overwritten during the localization process
#
#************************************************************
#
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements. See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership. The ASF licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
```

```
# with the License. You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.
#
#**************************************************************
```

7.2 Script for Generating SDF Files

The following script generates for each language a SDF file out of the PO-files.

⟨po2oo.sh⟩

```
#!/bin/bash
ScriptName="po2oo" #(.sh)
# Version 0.1.0
# This file should be in ~/local/bin/.

# If this script will be able to run on the Pootle server,
# you must replace 'DocRoot' with 'WorkDir'

DocRoot=/home/mechtilde/Projects/Git/ApacheOpenOffice/AOO-PootleWorkdir

# First, all language files are fetched from
# translate-ec2-va.apache.org to AOO-PootleWorkdir/PO-Files via Git on
# a specific date. All language files are then available.

for Language in ${LANGS[*]}
do
# Replace "_" with "-" for SDF directories
SDFLANG=$(echo ${Language} | sed -e 's/_/-/')
echo -e "\n"${Language}" "${SDFLANG}"\n" | tee --append ${LOG_FILE}
2>&1 echo -e "\n"${Language}" "${SDFLANG}"\n" >>${LOG_FILE}
```

The *SDF* files will be annotated with the license text.

⟨po2oo1⟩

```
mkdir -p ${DocRoot}/SDF-Files/${SDFLANG}/
cp ${DocRoot}/header.txt ${DocRoot}/SDF-Files/${SDFLANG}/localize.sdf
cd ${DocRoot}/PO-Files/${Language}/
```

Using *po2oo*:

po2oo [options] [--template=en-US.sdf] --languages=<targetlang> \ <input> <output>

targetlang is the ISO 639 code of the target language[2] (see Chapter 1.2, page 2).

2 https://de.wikipedia.org/wiki/ISO_639

input is the directory containing the translated PO files;

output is the SDF file generated from them.

The following options are used: *--template*, *--skipsource*, and *--keeptimestamp*. The template is a valid SDF file.

⟨po2oo2⟩

```
po2oo --progress=names \
--template=${DocRoot}/en-US.sdf \
--skipsource \
--keeptimestamp \
--language=${SDFLANG} \
${DocRoot}/PO-Files/${Language}/ \
${DocRoot}/SDF-Files/localize.${SDFLANG} 2>>${LOG_FILE}
```

The contents of the generated SDF file are inserted into the previously created *localize.sdf* file. After this merge, the (auxiliary) file generated by *po2oo* is removed.

⟨po2oo3⟩

```
cat ${DocRoot}/SDF-Files/localize.${SDFLANG} >>\
${DocRoot}/SDF-Files/${SDFLANG}/localize.sdf &&
rm ${DocRoot}/SDF-Files/localize.${SDFLANG}
done

echo "GSICHECK must be run in the build VM."
echo "The script there is called 'gsicheck.sh'."
```

Chapter 8

Running gsicheck

A prerequisite for running gsicheck is a functioning build environment.

This is also part of the source code

gsicheck -c -t -l de <path-to-language>/localize.sdf

-c outputs the line containing the error. This adds context to the error message.
-t means that only the translation is checked.
-l specifies the language in question.

⟨gsicheck.sh⟩

```
#!/bin/bash
ScriptName="gsicheck" #(.sh)
```

```
# Version 0.1.0
# This file should be in ~/local/bin/.
LOG=${HOME}/log_$(date +%Y%m%d%H%M%S).txt
touch ${LOG}

# This script is used on the VM to run GSICHECK
# after a build.

# Include the language data
. ${HOME}/configuration/langs.sh
echo ${LANGS} >>${LOG}

cd ${HOME}/openoffice/main/

for Language in
${LANGS[*]} do
solenv/bin/gsicheck -c -t -l ${Language}
../extras/l10n/source/${Language}/localize.sdf
echo ${Language}
read ok
done
```

Chapter 9

Performing Individual Steps

9.1 sdf to pot

As already described in Chapter 5.2 (page 14), the next step is to clean up the working directory.

Then, the *en-US.sdf* file will be converted into template files (POT files) (Chapter 5.2, page 14)

⟨sdf2pot.sh⟩

```
#!/bin/bash
ScriptName="sdf2pot" #(.sh)
# Version 0.1.0
# This file should be in ~/local/bin/.
TEMPLATES_DIR=/x1/www/po/aoo40/templates

# If the (non-empty) file en-US.sdf is in ${HOME}:
if [ -s ${HOME}/en-US.sdf ]
then
mv ${HOME}/en-US.sdf ${WORK_DIR}/en-US.sdf

# To be carefull
rm -r ${WORK_DIR}/POT-UI/
rm -r ${WORK_DIR}/POT-HELP/helpcontent2

mkdir -p ${WORK_DIR}/POT-UI
mkdir -p ${WORK_DIR}/POT-HELP

# Convert en-US.sdf to templates
${POOTLE_DIR}/python ${POOTLE_DIR}/oo2po --progress=verbose \
--pot ${WORK_DIR}/en-US.sdf ${WORK_DIR}/POT-UI/

# Move helpcontent2 to POT-HELP to separate HELP from UI
mv ${WORK_DIR}/POT-UI/helpcontent2 ${WORK_DIR}/POT-HELP/
```

```
fi
```

9.2 Importing POT Files into the Database

See also (Chapter 5.3, page 15)

⟨sync-pot-files2db.sh⟩

```
#!/bin/bash
ScriptName="sync-pot-files2db" #(.sh)

# Write *.pot files to the database
# This file should be in ~/local/bin/.

# Copy PotFiles-* into project directory
cp --recursive --verbose ${WORK_DIR}/POT-UI/* \
${PROJECT_PATH}/aoo40/templates/
cp --recursive --verbose ${WORK_DIR}/POT-HELP/* \
${PROJECT_PATH}/aoo40help/templates/

echo "Copied POT files into project directory." >>${LOG_FILE}

# Update the translation templates into the Pootle database

echo "Update UI stores" >>${LOG_FILE}
${POOTLE_DIR}/pootle update_stores -project=aoo40 --language=templates

echo "Update HELP stores" >>${LOG_FILE}
${POOTLE_DIR}/pootle update_stores -project=aoo40help \
--language=templates

echo "Update done" >>${LOG_FILE}

echo "All done"
```

9.3 Backing Up the Existing Translation

To do this, use the *sync-db2files.sh* script, which also performs this task during the nightly backup (see Chapter 6.1, page 26)

9.4 Synchronizing the translations with the source

see also Chapter 5.4, page 18

⟨pomerge.sh⟩

```
#!/bin/bash
ScriptName="pomerge" #(.sh)

# This script merges the recent PO files with the new template
# and creates new PO files.
# This file should be in ~/local/bin/.

# Reconcile languages with templates
for Language in ${LANGS}
do
```

```
echo "Pomerge UI "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pomerge \
--template=${PROJECT_PATH}/aoo40/templates/ \
--input ${WORK_DIR}/aoo40/${Language}/ \
--output ${PROJECT_PATH}/aoo40/${Language}/

echo "Pomerge HELP "${Language} >>${LOG_FILE}
echo $(date +%Y-%m-%d-%H%M%S) >>${LOG_FILE}

${POOTLE_DIR}/python ${POOTLE_DIR}/pomerge \
--template=${PROJECT_PATH}/aoo40help/templates/ \
--input ${WORK_DIR}/aoo40help/${Language}/ \
--output ${PROJECT_PATH}/aoo40help/${Language}/

done
```

9.5 Update the Database

The database at **translate.apache.org** is updated using the new PO files so that translators can update the translations.

⟨sync-po-files2db.sh⟩

```
#!/bin/bash
ScriptName="sync-po-files2db" #(.sh)

# Write *.po files into the database
# This file should be in ~/local/bin/.

# Reconcile languages with templates
for Language in ${LANGS[*]}
do
echo ${Language} >>${LOG_FILE}
${VIRTUAL_ENV}/bin/python ${VIRTUAL_ENV}/bin/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40 -language=${Language} &&

echo ${Language}"-Help" >>${LOG_FILE}

${VIRTUAL_ENV}/bin/python ${VIRTUAL_ENV}/bin/pootle \
--config ${VIRTUAL_ENV}/pootle.conf \
update_stores --project=aoo40help --language=${Language}
done
```

As result of running this script, the PO file and the database will be synchronised.

9.6 Updating the Working Directory

Finally, the working directory in the user's home directory is also updated by copying the PO files there using the `cp2home.sh` script (Chapter 6.2, page 28) and versioning them with Git.

Chapter 10

Rarely Used Scripts

This section documents the scripts that are only needed when setting up a new server.

- Creating Language Directories
- Recovery After a Failed Sync

10.1 Creating Language Directories

The following script creates language directories on the Pootle server.

⟨mklangdirs.sh⟩

```
#!/bin/bash

# You can use this script to create all language
# directories in /x1/www/po/ on a new server.
# (if not exist)

cd ${PROJECT_PATH}/aoo40

mkdir --verbose ${LANGS}

ls -la
echo "Done in /x1/www/po/aoo40"

cd ${PROJECT_PATH}/aoo42help

mkdir --verbose $(find ${WORK_DIR}/PO-Files/ -name '*' \
| sed --expression='s/\/home\/mechtilde\/aoo\/workdir\/PO-Files\///' \
| grep helpcontent2 | sed --expression='s/\/[a-z 0-9]*.po$//')
```

There is a directory named `*helpcontent2*` for the help text, located under the language directory. In `*x1/www/po*`, the help text is located in the `*aoo40help*` directory.

⟨mklangsdirs1⟩

```
ls -la

echo "Done in /x1/www/po/aoo40help"

echo -e 'Please become root and adjust the permissions\nusing "chown -R www-
data ."'
```

10.2 Recovery After a Failure

⟨reset-pootle.sh⟩

```
#!/bin/bash
ScriptName="reset-pootle" #(.sh)
# Version 0.1.0
# This file should be in ~/local/bin/

# You can use this script to restore Pootle
# e.g., after an incorrect import of templates
# It does it from home directory into the project path

for Language in ${LANGS}
do
echo ${Language} >>${LOG_FILE}
cp --recursive --verbose ${WORK_DIR}/PO-Files/${Language} \
${PROJECT_PATH}/aoo40

rm --recursive --verbose \
${PROJECT_PATH}/aoo40help/${Language}/helpcontent2

mv --verbose ${PROJECT_PATH}/aoo40/${Language}/helpcontent2 \
${PROJECT_PATH}/aoo40help/${Language}
done

echo "You should run sync-po-files2db.sh now!"
```

After that, the `sync-po-files2db.sh` script is called.

10.3 Restoring the PO Files

The following script, `sync-po-files.sh`, uses the PO files in the working directory to restore the PO files on the Pootle server. It can also be used to create the PO files for all languages there. Only files on the Pootle server with an older date than those in the working directory will be updated.

⟨sync-po-files.sh⟩

```
#!/bin/bash
ScriptName="sync-po-files" #(.sh)

# You can use this script to create all language
# directories in /x1/www/po/ on a new Pootle server
# (if  not exist) and sync *.po files.

# Go to the directory where the *.po files are in subdirectories
cd ${WORK_DIR}/PO-Files

# Create a list of all *.po files in the subdirectories helpcontent2
find . -name '*' | grep helpcontent2  >helpcontent2.txt

# Copy all files from list to aoo42help
```

```
rsync --verbose --files-from=helpcontent2.txt . ${PROJECT_PATH}/aoo42help/

echo "Done in "${PROJECT_PATH}"/aoo42help"

rm --verbose helpcontent2.txt

# Copy all other *.po files
rsync --archive --verbose --exclude='*helpcontent2*' . ${PROJECT_PATH}/aoo42

echo "Done in "${PROJECT_PATH}"/aoo42"


echo -e 'Please become root and adjust the permissions\nusing "chown -R www-
data ."'
```

Chapter 11

Extraction Scripts

11.1 Extracting the Documentation

⟨create-pdf.sh⟩

```
#!/bin/bash
# With this script, you can create a *.pdf and a *.epub file
# from a *.nw file.
# In line 6 you can find the file name.

FileName=TranslationAOO
noweave -index -delay ${FileName}.nw >${FileName}.tex
noweave -filter l2h -index -html ${FileName}.nw | htmltoc >${FileName}.html
[ ! -f ${FileName}.idx ] || makeindex ${FileName}.idx

for ((i=4; i>0; i--))
do
echo $i
pdflatex ${FileName}.tex
done
tex4ebook -f epub ${FileName}.tex
```

11.2 Extraction of the documented scripts

⟨create-scripts⟩

```
# Now it extracts the scripts

set -e

scripts="

LanguageLists.sh
UpdateTranslation.sh
mnrun.sh
sync-db2files.sh
cp2home
header.txt
po2oo.sh
```

```
Download.sh
sdf2pot.sh
sync-pot-files2db.sh
reset-pootle.sh
sync-po-files.sh
"

FileName=TranslationAOO

t='date +%c'

function build_script()
{
notangle -R"$script" ${FileName}.nw >"$script"

sed --in-place \
--expression='s/This is the end, my friend[[:cntrl:]]*/This is the end, my friend/' \
"$1"
echo "#generated on $t" >>"$1"
chmod ugo+x "$1"
bash -n "$1"
}

for script in $scripts;
do build_script "$script"
done
```

Chapter 12

Scripts for Automation

12.1 Pootle.conf

⟨Pootle.conf⟩

```
# -*-coding: utf-8 -*-
"""Sample configuration file.

This file contains the settings that administrators are likely to change.
You can find the defaults in the ``*.conf`` files for more advanced settings.
"""

from pootle.i18n.gettext import ugettext_lazy as _

#
# Base
#

# Site title

POOTLE_TITLE = _(u'Pootle Translation Server')

# Local time zone for this installation. Choices can be found here:
# https://en.wikipedia.org/wiki/List_of_tz_zones_by_name
# although not all options may be available on all operating systems.
# If running in a Windows environment, this must be set to the same as
# your system time zone.
TIME_ZONE = 'UTC'

# Make this unique, and do not share it with anyone.
```

```
# SECRET_KEY = '5B9vwlJXdCp/DR0i8i8F2VmyKEKqeTKlnXovP9NLYfbJDdqrnTWhFmRUaCTahthEjNA='
SECRET_KEY = 'FpuS2TZuWjID9XShjBN8ngivUPqo30QemCBJgH1lDd2fjyj9GtEYCdmisN5guDZFEgs='

# A list of strings representing the host/domain names that this Pootle server
# can serve. This is a Django security measure.
ALLOWED_HOSTS = [
'127.0.0.1',
'localhost',
'translate.apache.org',
'translate-ec2-va.apache.org',
#'${your_server}',
]

# Set this to a canonical URL for your site *without a trailing slash*
# This will be used when deriving URLs to send out emails
# If you use the `django.contrib.sites` framework, set this to blank
POOTLE_CANONICAL_URL = "https://translate.apache.org"

#
# Backends
#

# Database backend settings
DATABASES = {
'default': {
# Replace 'sqlite3' with 'postgresql' or 'mysql'.
# 'ENGINE': 'django.db.backends.sqlite3',
'ENGINE': 'django.db.backends.mysql',
# Database name or path to the database file if using sqlite3.
# 'NAME': working_path('dbs/pootle.db'),
'NAME': 'pootledb',
# Not used with sqlite3.
# 'USER': '',
'USER': 'pootle',
# Not used with sqlite3.
# 'PASSWORD': "",
'PASSWORD': 'x2vv5YbYjyOL16vrR504pdjJz+bJd5R0',
# Set to an empty string for localhost. Not used with sqlite3.
# 'HOST': '',
'HOST': '127.0.0.1',
# Set to empty string for localhost. Not used with sqlite3.
'PORT': '',
# See https://docs.djangoproject.com/en/1.10/topics/db/transactions/
# required for Django + SQLite
'ATOMIC_REQUESTS': True,
#'OPTIONS': {
# # MySQL:
# # https://docs.djangoproject.com/en/1.10/ref/databases/#setting-sql-mode
# 'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
#},
'TEST': {
'NAME': 'test_pootle',
}
}
}

# Cache backend settings
# For more information, see
# https://docs.djangoproject.com/en/1.10/topics/cache/#setting-up-the-cache
# and http://niwibe.github.io/django-redis/
#CACHES = {
#     'default': {
#           'BACKEND': 'django_redis.cache.RedisCache',
#           'LOCATION': 'redis://127.0.0.1:6379/1',
```

```
#             'TIMEOUT': 60,
#       },
#       'redis': {
#             'BACKEND': 'django_redis.cache.RedisCache',
#             'LOCATION': 'redis://127.0.0.1:6379/2',
#             'TIMEOUT': None,
#       },
#       'lru': {
#             'BACKEND': 'django_redis.cache.RedisCache',
#             'LOCATION': 'redis://127.0.0.1:6379/3',
#       'TIMEOUT': 604800, # 1 week
#       },
#       'exports': {
#             'BACKEND': 'django.core.cache.backends.filebased.FileBasedCache',
#             'LOCATION': working_path('exports/'),
#       'TIMEOUT': 259200, # 3 days.
#       },
#}


#
# Local Translation Memory service
#

# Setup for connection to the ElasticSearch server for translation memory
# based on the projects hosted on Pootle.
# You may want to set AMAGAMA_URL to '' (empty string) if using this service.
#
#POOTLE_TM_SERVER = {
#       'local': {
#             'ENGINE': 'pootle.core.search.backends.ElasticSearchBackend',
#             'HOST': 'localhost',
#             'PORT': 9200,
#             # Every TM server must have its own unique index.
#             'INDEX_NAME': 'translations',
#             # Provides a weighting factor to alter the final score for TM results
#             # from this TM server. Valid values are between "0.0" and "1.0",
#             # both included. Defaults to "1.0" if not provided.
#             'WEIGHT': 1.0,
#       },
#       'external': {
#             'ENGINE': 'pootle.core.search.backends.ElasticSearchBackend',
#             'HOST': 'localhost',
#             'PORT': 9200,
#             'INDEX_NAME': 'external-translations',
#             'WEIGHT': 0.9,
#       },
#}

#
# Logging
#

# The directory where Pootle writes its logs
POOTLE_LOG_DIRECTORY = '/x1/www/pootle/log'

#
# Site
#

# This Pootle server admins
ADMINS = (
('Chris Lambertus','cml@apache.org '),
# ('Your Name', 'your_email@domain.com '),
)

# Mail settings
```

```
# Default email address to use for messages sent by Pootle.
DEFAULT_FROM_EMAIL = 'translate@apache.org '

# Address to receive messages sent via the contact form.
POOTLE_CONTACT_ENABLED = False
POOTLE_CONTACT_EMAIL = 'root@apache.org '

# Email address to report string errors to, unless a report email was set for
# the project for which the string error is being reported.
# Use this to forward string error reports to an address
# different from the general contact address.
POOTLE_CONTACT_REPORT_EMAIL = 'users@infra.apache.org '

# Whether to email reviewers' feedback to suggesters.
POOTLE_EMAIL_FEEDBACK_ENABLED = False

# Mail server settings

# By default, Pootle uses the SMTP server on localhost. If the server is not
# configured for sending emails, uncomment and use these settings to set up an
# external outgoing SMTP server.

# Example for Google as an external SMTP server
#EMAIL_HOST_USER = 'USER@YOUR_DOMAIN.com'
#EMAIL_HOST_PASSWORD = 'YOUR_PASSWORD'
#EMAIL_HOST = 'smtp.gmail.com'
#EMAIL_PORT = 587
#EMAIL_USE_TLS = True
#

#
Translation
#

# The directory where the translation files are stored
# POOTLE_TRANSLATION_DIRECTORY = working_path('translations')
POOTLE_TRANSLATION_DIRECTORY = '/x1/www/po'

# A two-tuple defining the markup filter to apply in certain text areas.
#
# -Accepted values for the first element are 'markdown' and
#      'html' (deprecated).
# -The second element should be a dictionary of keyword arguments that will be
#     passed to the markup function.
#
POOTLE_MARKUP_FILTER = ('markdown', {})

# Set the backends you want to use to enable translation suggestions through
# several online services. To disable this feature completely just comment all
# the lines to set an empty list [] for the POOTLE_MT_BACKENDS setting.
#
# The second parameter for each backend option is the API key, which will
# be used if the service supports the use of an API key.
#
# Available options are:
# 'CAIGHDEAN_TRANSLATE': Caighdeán Translate service.
#            Provides translation from Manx Gaelic (gv) and
#            Scottish Gaelic (gd) into Irish Gaelic (ga-IE)
#            See more at https://github.com/kscanne/caighdean
# 'GOOGLE_TRANSLATE': Google Translate service.
#            To use this service, you need to set the API key.
#            Note that the Google Translate API is a paid service
#            See more at https://cloud.google.com/translate/v2/pricing
```

```
# 'WELSH_TRANSLATE': Welsh translation service.
#             Provides translation from English (en) to Welsh (cy)
#             You need an API key to use this service.
#             See more at http://techiaith.cymru/api/translation/?lang=en
# 'YANDEX_TRANSLATE': Yandex Translate service.
#             To use this service, you need to set the API key.
#             See more at https://tech.yandex.com/translate/
#
POOTLE_MT_BACKENDS = [
#      ('CAIGHDEAN_TRANSLATE', ''),
#      ('GOOGLE_TRANSLATE', ''),
#      ('WELSH_TRANSLATE', ''),
#      ('YANDEX_TRANSLATE', ''),
]

# Custom template directories
#
TEMPLATES[0]['DIRS'] = [
# working_path(os.path.join('custom', 'templates')),
]

POOTLE_SIGNUP_ENABLED = False
AUTH_LDAP_USER_QUERY_FIELD = 'username'
AUTH_LDAP_ALWAYS_UPDATE_USER = True
AUTH_LDAP_SERVER_URI = 'ldaps://ldap-master.apache.org'
AUTH_LDAP_BASE_DN = 'ou=people,dc=apache,dc=org'
AUTH_LDAP_BIND_DN = 'cn=nss_ldap,ou=users,ou=services,dc=apache,dc=org'
AUTH_LDAP_BIND_PASSWORD = 'b1t3me'
AUTH_LDAP_USER_DN_TEMPLATE = 'uid=%(user)s,ou=people,dc=apache,dc=org'
AUTHENTICATION_BACKENDS = (
'django_auth_ldap.backend.LDAPBackend',
'django.contrib.auth.backends.ModelBackend',
)
AUTH_LDAP_USER_ATTRLIST = ["*", "+"]
AUTH_LDAP_USER_ATTR_MAP = {
'first_name': 'givenName',
'last_name': 'sn',
'email': 'asf-committer-email',
'username': 'uid',
'full_name': 'cn',
}
```

References

https://ci2.apache.org/#/builders/70/builds/43

https://svn.apache.org/repos/infra/infrastructure/buildbot2/projects/openofficeorg.py

https://ci2.apache.org/#/builders/70

https://nightlies.apache.org/openoffice/install/linsnap-42x/?C=M;O=D

https://github.com/translate/pootle/issues/6902

https://github.com/mozilla/pontoon

https://github.com/WeblateOrg/weblate/discussions